E4 Educator v2.0

T08

TFT display and sprite pipeline

Tier 2 · Tutorial 1 of 9

Walark Electronics · Fundrum Engineering · May 2026

Welcome to Tier 2

T08 is the biggest step up in the course. Everything from Tier 1 applies here. The TFT is faster, larger, and more capable than the OLED — and the sprite pipeline is how professional embedded display firmware is written.

In this tutorial

You will learn:

- How the ILI9341 TFT display works and how it differs from the OLED
- How TFT_eSPI is configured using build_flags instead of User_Setup.h
- How to initialise the display and control the backlight correctly
- The RGB565 colour format and how to calculate and use colours
- How to draw text, shapes, and filled regions directly to the TFT
- Why direct drawing causes flicker and how sprites solve it
- The sprite double-buffer pipeline — the professional approach to TFT updates
- How to design and implement a multi-zone TFT dashboard layout
- How to use smooth fonts loaded from the TFT_eSPI font pipeline

You will need:

- E4 Educator v2.0 board with TFT display socketed and ESP32 fitted
- T01 through T07 completed — Button.h and TonePlayer.h available
- USB-C cable (data-capable)

1 The ILI9341 TFT display

The 2.8" ILI9341 TFT is a full-colour LCD display with 240 pixels wide by 320 pixels tall. Unlike the OLED which communicates over I2C, the TFT uses SPI — a four-wire bus that transfers data significantly faster, allowing smooth animation and colour updates.

1.1 TFT versus OLED — the key differences

Property	0.96" OLED	2.8" ILI9341 TFT
Resolution	128×64 monochrome	240×320 16-bit colour
Interface	I2C (2 wires)	SPI (4+ wires)
Colours	White on black only	65,536 colours (RGB565)
Backlight	Self-lit OLED pixels	LED backlight via GPIO 12 (TFT_BL)
Config	Library address 0x3C	build_flags pin definitions — no User_Setup.h needed
Additional	None	XPT2046 touch + microSD on same module
Course tier	Tier 1 (T05)	Tier 2 (T08 onwards)

1.2 The SPI bus and E4 pin assignments

Signal	GPIO / Define	Purpose
MOSI	`23 / TFT_MOSI`	Master Out Slave In — data from ESP32 to TFT
SCLK	`18 / TFT_SCLK`	SPI clock generated by ESP32
MISO	`19 / TFT_MISO`	Master In Slave Out — data from TFT to ESP32 (touch and SD read)
TFT_CS	`5 / TFT_CS`	TFT chip select — LOW to address TFT
DC	`17 / TFT_DC`	Data/Command — HIGH = data, LOW = command byte
BL	`12 / TFT_BL`	Backlight PWM via BC847 transistor. Init LOW before LEDC.

1.3 The display orientation

The ILI9341 can be used in landscape (320 wide × 240 tall) — the E4 Educator always uses landscape orientation. The E4 board mounts the TFT in portrait orientation. All zone constants in this tutorial assume portrait mode.

```
// Portrait mode (E4 default)
tft.setRotation(0);   // 0 = landscape on E4 Educator v2.0
// Screen width:  240  (tft.width())
// Screen height: 240  (tft.height())

// Landscape mode if needed
tft.setRotation(1);   // 1 = landscape, origin top-left
// Screen width:  320
// Screen width:  320  (tft.width())
```

2 TFT_eSPI — configuration via build_flags

TFT_eSPI is the library used for all TFT display work in E4 projects. It is highly optimised for the ESP32 and supports sprites, smooth fonts, and hardware SPI acceleration. On most projects, TFT_eSPI requires editing a User_Setup.h file inside the library folder. On the E4, we configure it entirely via build_flags instead — keeping the library unmodified and all pin definitions in one place.

2.1 platformio.ini for T08

```
[env:e4_educator_v2]
platform   = espressif32@6.6.0
board      = esp32dev
framework  = arduino
monitor_speed = 115200
monitor_port  = COM10
upload_port   = COM10

lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO

build_flags =
  -DFW_VERSION=\"1.0.0\"
  -DFW_DATE=\"2026-05-01\"
  ; E4 v2.0 pin definitions
  -DLED_RED=16 -DLED_GREEN=26 -DLED_BLUE=32
  -DBTN_NEXT=13 -DBTN_ENT=14
  -DTFT_CS=5 -DTFT_DC=17 -DTFT_BL=12
  -DTFT_MOSI=23 -DTFT_SCLK=18 -DTFT_MISO=19
  -DTOUCH_CS=33 -DSD_CS=15
  -DI2C_SDA=21 -DI2C_SCL=22
  -DONE_WIRE=4 -DLDR=39 -DMIC_ANALOG=36
  -DMEMS_BCLK=32 -DMEMS_WS=35 -DMEMS_SD=34
  -DDAC_AUDIO=25 -DPIEZO=27
  ; TFT_eSPI driver configuration via build_flags
  -DUSER_SETUP_LOADED=1
  -DILI9341_DRIVER=1
  -DTFT_WIDTH=320
  -DTFT_HEIGHT=240
  -DTFT_MISO=19
  -DTFT_MOSI=23
  -DTFT_SCLK=18
  -DTFT_CS=5
  -DTFT_DC=17
  -DTFT_RST=-1
  -DLOAD_GLCD=1
  -DLOAD_FONT2=1
  -DLOAD_FONT4=1
  -DLOAD_FONT6=1
  -DLOAD_FONT7=1
  -DLOAD_FONT8=1
  -DLOAD_GFXFF=1
  -DSMOOTH_FONT=1
```

```
  -DSPI_FREQUENCY=27000000
  -DSPI_READ_FREQUENCY=20000000
  -DSPI_TOUCH_FREQUENCY=2500000
```

Pin the TFT_eSPI version

The @ 2.5.43 version pin on the TFT_eSPI lib_deps line is critical. TFT_eSPI updates frequently and breaking changes between versions are common. Pinning ensures your project builds identically on any machine and never breaks from an upstream update. Always pin library versions in production E4 projects.

2.2 Why USER_SETUP_LOADED=1 matters

When TFT_eSPI sees -DUSER_SETUP_LOADED=1 in the build flags, it skips reading its own User_Setup.h file entirely and uses only the defines provided by build_flags instead. This is the key that makes the build_flags approach work — without it, the library would still try to read its configuration file and cause conflicts.

Key point

The combination of -DUSER_SETUP_LOADED=1 plus the ILI9341_DRIVER and pin defines gives TFT_eSPI everything it needs from platformio.ini alone. The library folder is never modified. This means the library can be updated (carefully, with version pinning) without losing your pin configuration.

3 RGB565 colour

The ILI9341 uses RGB565 colour format — each pixel is stored as a 16-bit value (uint16_t) with 5 bits for red, 6 bits for green, and 5 bits for blue. This gives 65,536 possible colours.

3.1 Built-in colour constants

TFT_eSPI provides a set of common colour constants ready to use:

Constant	RGB565 value	Colour
TFT_BLACK	0x0000	Black
TFT_WHITE	0xFFFF	White
TFT_RED	0xF800	Red
TFT_GREEN	0x07E0	Green
TFT_BLUE	0x001F	Blue
TFT_YELLOW	0xFFE0	Yellow
TFT_CYAN	0x07FF	Cyan
TFT_MAGENTA	0xF81F	Magenta
TFT_ORANGE	0xFDA0	Orange
TFT_DARKGREY	0x7BEF	Dark grey

3.2 Custom colours

To create a custom colour from RGB values (0-255 each), use the tft.color565() function:

```
// Convert standard RGB (0-255 each) to RGB565
uint16_t myBlue    = tft.color565(0,   102, 204);  // Walark brand blue
uint16_t myOrange  = tft.color565(255, 140,   0);  // Dashboard accent
uint16_t myDarkBg  = tft.color565(20,   20,  30);  // Near-black background

// Or calculate directly in Python for build_flags constants:
// ((r>>3)<<11) | ((g>>2)<<5) | (b>>3)
// For r=0, g=102, b=204:
// ((0>>3)<<11) | ((102>>2)<<5) | (204>>3)
// =  0 | (25<<5) | 25 = 0 | 800 | 25 = 0x0339

// Define your project palette at the top of main.cpp
#define COL_BG      0x0000   // Background — black
#define COL_HEADER  0x0339   // Header — brand blue
#define COL_TEXT    0xFFFF   // Primary text — white
#define COL_ACCENT  0xFD20   // Accent — orange
#define COL_OK      0x07E0   // Status OK — green
#define COL_WARN    0xFFE0   // Status warning — yellow
#define COL_ERR     0xF800   // Status error — red
```

Key point

Define your entire colour palette as #define constants at the top of main.cpp. Never use raw RGB565 hex values scattered through drawing code. When you want to change the colour scheme you change one block of defines and everything updates. This is the same discipline as GPIO defines in build_flags.

4 Direct drawing to the TFT

TFT_eSPI provides a rich set of drawing functions. Understanding them is essential before introducing sprites — you need to know what sprites replace and why.

4.1 Basic drawing functions

```
#include <Arduino.h>
#include <TFT_eSPI.h>

TFT_eSPI tft;

void setup() {
  Serial.begin(115200);

  // Backlight init — always LOW first
  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);

  // Initialise TFT
  tft.init();
  tft.setRotation(0);          // Landscape — correct for E4 Educator v2.0
  tft.fillScreen(TFT_BLACK);   // Clear to black

  // Backlight on at full brightness
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 255);

  // ── Shapes ──────────────────────────────────────────────
  tft.fillRect(0, 0, 240, 40, COL_HEADER);       // Header bar
  tft.drawRect(10, 60, 100, 80, TFT_WHITE);      // Rect outline
  tft.fillRect(130, 60, 100, 80, TFT_DARKGREY);  // Filled rect
  tft.drawCircle(60, 180, 40, TFT_CYAN);         // Circle outline
  tft.fillCircle(180, 180, 40, TFT_ORANGE);      // Filled circle
  tft.drawLine(0, 260, 240, 260, TFT_WHITE);     // Horizontal line

  // ── Text ────────────────────────────────────────────────
  tft.setTextColor(TFT_WHITE, COL_HEADER);  // Text + bg colour
  tft.setTextSize(2);
  tft.setCursor(10, 10);
  tft.print("E4 Educator v2.0");

  tft.setTextColor(TFT_YELLOW, TFT_BLACK);
  tft.setTextSize(3);
  tft.setCursor(10, 280);
  tft.print("Hello TFT!");
}

void loop() {}
```

setTextColor() requires two arguments on TFT

Unlike the OLED library, TFT_eSPI's setTextColor() should always be called with both text colour and background colour: tft.setTextColor(textCol, bgCol). Without the background colour, text is drawn transparently over whatever is behind it — this causes leftover pixels when values change and is a very common source of display corruption.

4.2 The flicker problem

When you update a value on the TFT directly — erase the old text with a filled rectangle, then draw the new text — you see visible flicker. The display shows black momentarily between erase and redraw. At low update rates this is tolerable. At faster rates, or with larger areas, it is very noticeable and unprofessional.

This is the problem sprites solve.

5 The sprite double-buffer pipeline

A sprite in TFT_eSPI is an off-screen buffer — a region of RAM that you draw into invisibly, then push to the TFT in one fast operation. Because the push is a single DMA transfer, the display never shows a partially-drawn state. Flicker is eliminated.

The double-buffer pipeline works like this:

Step	Action
1	Fill the sprite with the background colour — erasing everything from the previous frame
2	Draw all text, shapes, and values into the sprite — nothing is visible yet
3	Push the sprite to the TFT at the correct X/Y position — the entire zone updates atomically in one transfer

Key point

The sprite push is the only moment the display changes. Steps 1 and 2 happen entirely in RAM, invisibly. This is why there is no flicker — the user never sees an intermediate state. This is the same technique used in game graphics and professional embedded UI frameworks.

5.1 Creating and using a sprite

```
#include <Arduino.h>
#include <TFT_eSPI.h>

TFT_eSPI tft;
TFT_eSprite spr = TFT_eSprite(&tft);  // Sprite attached to tft

// Zone dimensions
#define ZONE_W  240
#define ZONE_H   60
#define ZONE_X    0
#define ZONE_Y  130

void setup() {
  Serial.begin(115200);

  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
  tft.init();
  tft.setRotation(0);
  tft.fillScreen(TFT_BLACK);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 255);

  // Create the sprite — allocates RAM
  spr.setColorDepth(16);           // 16-bit colour (RGB565)
  spr.createSprite(ZONE_W, ZONE_H);
  Serial.println("Sprite created.");
}
```

```
int counter = 0;
unsigned long lastUpdate = 0;

void loop() {
  if (millis() - lastUpdate >= 500) {
    lastUpdate = millis();
    counter++;

    // Step 1: fill sprite with background
    spr.fillSprite(TFT_BLACK);

    // Step 2: draw into sprite
    spr.setTextColor(TFT_WHITE, TFT_BLACK);
    spr.setTextSize(2);
    spr.setCursor(10, 5);
    spr.print("Count:");
    spr.setTextSize(4);
    spr.setCursor(10, 28);
    spr.print(counter);

    // Step 3: push sprite to TFT — no flicker
    spr.pushSprite(ZONE_X, ZONE_Y);
  }
}
```

Run this and watch the counter increment every 500ms. The number updates cleanly with no flicker — even though it is changing on screen. This is the sprite pipeline working.

5.2 Sprite RAM usage

Every sprite occupies RAM: width × height × 2 bytes (for 16-bit colour). The ESP32 has 320KB of RAM total. Large sprites eat into this quickly.

Sprite size	Width × Height	RAM used	Notes
Full screen	240×320	150KB	Too large — leaves little for stack and heap
Half screen	240×160	75KB	Workable for large zones
Zone sprite	240×60	28KB	Ideal for individual dashboard zones
Value sprite	160×40	12KB	Minimal sprite for a single value update

The best strategy for a dashboard is one sprite sized to the largest zone, reused for each zone update. Create it once in setup(), never delete it. This avoids repeated allocation and deallocation which fragments heap memory over time.

6 Smooth fonts

TFT_eSPI supports smooth (anti-aliased) fonts via the VLW font format. These look dramatically better than the built-in bitmap fonts — edges are smooth rather than blocky, and they scale to any size without the pixel staircase effect.

6.1 Using smooth fonts from the filesystem

VLW font files are stored on LittleFS (covered in T13) and loaded at runtime. For T08, we use the built-in GFX fonts and the TFT_eSPI bitmap fonts to establish the technique. Smooth fonts are introduced properly in T13 once LittleFS is established.

For now, the built-in fonts provide four useful sizes:

Font	Approx height	Best use
`setTextSize(1)`	8px	Small labels, status text — legible but compact
`setTextSize(2)`	16px	Standard body text, sensor labels
`setTextSize(3)`	24px	Values, headings
`setTextSize(4)`	32px	Large prominent values
`loadFont() VLW`	Any	Smooth anti-aliased fonts — introduced in T13

7 Multi-zone TFT dashboard

A zone layout for the TFT follows exactly the same principles as the OLED zone layout in T05 — define Y positions as constants, draw static chrome once, update only changed zones using sprites. The difference is colour, resolution, and the sprite pipeline replacing direct draws.

7.1 E4 standard dashboard layout

This is the standard three-zone landscape layout used throughout Tier 2 and Tier 3 projects:

Zone	Y range	Height	Content
Header	`Y 0–34`	35px	Title bar — project name, firmware version, status icon
Content	`Y 35–199`	165px	Main content area — sensor values, charts, lists
Footer	`Y 200–239`	40px	Status bar — uptime, page indicator, mode flags

7.2 Full dashboard project

Create a new project called E4_T08_Dashboard. Copy Button.h into src/. This project reads the AHT20 and displays a colour-coded temperature and humidity dashboard on the TFT using the sprite pipeline.

```
#include <Arduino.h>
#include <Wire.h>
#include <TFT_eSPI.h>
#include <Adafruit_AHTX0.h>
#include "Button.h"

// ── Colour palette ──────────────────────────────────────
#define COL_BG      0x0000
#define COL_HEADER  0x0319   // Dark brand blue
#define COL_FOOTER  0x18C3   // Slightly lighter blue
#define COL_TEXT    0xFFFF   // White
#define COL_LABEL   0xC618   // Light grey
#define COL_ACCENT  0xFD20   // Orange
#define COL_OK      0x07E0   // Green
#define COL_WARN    0xFFE0   // Yellow
#define COL_ERR     0xF800   // Red

// ── Zone layout ─────────────────────────────────────────
#define ZONE_HEADER_Y   0
#define ZONE_HEADER_H  40
#define ZONE_CONTENT_Y 35
#define ZONE_CONTENT_H 165
#define ZONE_FOOTER_Y  280
#define ZONE_FOOTER_H  40
#define SCR_W          320

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);
```

```
Adafruit_AHTX0 aht;
Button nextBtn(BTN_NEXT);
Button entBtn(BTN_ENT);

float tempC = 0.0, humidity = 0.0;
unsigned long lastRead   = 0;
unsigned long startTime  = 0;
const unsigned long READ_INTERVAL = 3000;

// ── Backlight ──────────────────────────────────────────────
void initBacklight(int brightness = 255) {
  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, brightness);
}

// ── Draw static chrome (runs once) ─────────────────────────
void drawChrome() {
  tft.fillScreen(COL_BG);

  // Header bar
  tft.fillRect(0, ZONE_HEADER_Y, SCR_W, ZONE_HEADER_H, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("E4 Env Dashboard");

  // Footer bar
  tft.fillRect(0, ZONE_FOOTER_Y, SCR_W, ZONE_FOOTER_H, COL_FOOTER);
  tft.setTextColor(COL_LABEL, COL_FOOTER);
  tft.setTextSize(1);
  tft.setCursor(8, 292);
  tft.printf("FW: %s  |  T08", FW_VERSION);

  // Content zone dividers
  tft.drawFastHLine(20, 160, 200, COL_LABEL);  // Divide temp/humid

  // Static labels
  tft.setTextColor(COL_LABEL, COL_BG);
  tft.setTextSize(2);
  tft.setCursor(8, 48);
  tft.print("TEMPERATURE");
  tft.setCursor(8, 172);
  tft.print("HUMIDITY");
}

// ── Update a value zone with sprite ────────────────────────
void updateTempZone(float temp) {
  uint16_t valCol = (temp > 28.0) ? COL_ERR :
                    (temp > 24.0) ? COL_WARN : COL_OK;

  spr.fillSprite(COL_BG);
```

```
  spr.setTextColor(valCol, COL_BG);
  spr.setTextSize(4);
  spr.setCursor(8, 10);
  spr.printf("%.1f", temp);
  spr.setTextSize(2);
  spr.setCursor(8, 58);
  spr.setTextColor(COL_LABEL, COL_BG);
  spr.print("degrees C");
  spr.pushSprite(0, 70);
}

void updateHumidZone(float humid) {
  uint16_t valCol = (humid > 70.0) ? COL_ERR :
                    (humid > 60.0) ? COL_WARN : COL_OK;

  spr.fillSprite(COL_BG);
  spr.setTextColor(valCol, COL_BG);
  spr.setTextSize(4);
  spr.setCursor(8, 10);
  spr.printf("%.1f", humid);
  spr.setTextSize(2);
  spr.setCursor(8, 58);
  spr.setTextColor(COL_LABEL, COL_BG);
  spr.print("percent RH");
  spr.pushSprite(0, 192);
}

void updateFooter() {
  unsigned long uptimeSec = (millis() - startTime) / 1000;
  spr.fillSprite(COL_FOOTER);
  spr.setTextColor(COL_LABEL, COL_FOOTER);
  spr.setTextSize(1);
  spr.setCursor(8, 12);
  spr.printf("Up: %lus  |  NEXT=refresh", uptimeSec);
  spr.pushSprite(0, ZONE_FOOTER_Y);
}

void readAndUpdate() {
  sensors_event_t h, t;
  aht.getEvent(&h, &t);
  tempC    = t.temperature;
  humidity = h.relative_humidity;
  updateTempZone(tempC);
  updateHumidZone(humidity);
  updateFooter();
  Serial.printf("Temp: %.1fC  Humidity: %.1f%%\n", tempC, humidity);
}

void setup() {
  Serial.begin(115200);
  Serial.printf("T08 TFT Dashboard  FW:%s\n", FW_VERSION);

  Wire.begin(I2C_SDA, I2C_SCL);
```

```
  pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
  pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);
  nextBtn.begin();
  entBtn.begin();

  initBacklight(200);   // 80% brightness
  tft.init();
  tft.setRotation(0);

  // Create sprite — sized to content zone width x row height
  spr.setColorDepth(16);
  spr.createSprite(SCR_W, 80);  // Reused for both value zones

  bool ok = true;
  if (!aht.begin(&Wire)) {
    Serial.println("ERROR: AHT20"); ok = false;
  }
  if (!ok) { digitalWrite(LED_RED, HIGH); while(true){delay(100);} }

  startTime = millis();
  drawChrome();
  readAndUpdate();
  digitalWrite(LED_GREEN, HIGH);
  Serial.println("Ready.");
}

void loop() {
  unsigned long now = millis();
  Button::Event nextEv = nextBtn.read();

  if (nextEv == Button::SHORT_PRESS) {
    lastRead = now;
    readAndUpdate();
  }

  if (now - lastRead >= READ_INTERVAL) {
    lastRead = now;
    readAndUpdate();
  }

  // Update footer uptime every second
  static unsigned long lastFooter = 0;
  if (now - lastFooter >= 1000) {
    lastFooter = now;
    updateFooter();
  }
}
```

7.3 What to observe

- The TFT shows a clean three-zone dashboard — dark header, large colour-coded values, uptime footer
- Temperature value colour changes: green (normal) → yellow (warm) → red (hot)

- Humidity value colour changes: green (normal) → yellow (high) → red (very high)
- Values update every 3 seconds with zero flicker — sprite pipeline at work
- Footer uptime counter increments every second independently of sensor reads
- NEXT short press forces an immediate sensor read
- Breathe on the AHT20 and watch both temperature and humidity respond with colour changes

Try this

Change initBacklight(200) to initBacklight(50) and observe the difference in brightness. Then change the COL_HEADER colour constant from 0x0319 to 0xF800 and rebuild — the entire header turns red with one constant change. This is why the colour palette approach matters.

8 T08 summary

Concept	Key takeaway
ILI9341 TFT	240×320, 16-bit colour, SPI bus, separate backlight pin. socketed on E4 — OLED remains on I2C bus.
TFT_eSPI config	USER_SETUP_LOADED=1 plus driver and pin defines in build_flags. No User_Setup.h file. Version pinned at 2.5.43.
Backlight init	Always pinMode(TFT_BL, OUTPUT) and digitalWrite LOW before ledcSetup(). GPIO 12 boot sensitivity.
RGB565 colour	tft.color565(r, g, b) converts 0-255 RGB to RGB565. Define your palette as named constants. Never scatter raw hex values through code.
setTextColor()	Always provide both text and background colour on TFT: tft.setTextColor(textCol, bgCol). Background prevents transparent text corruption.
Sprite pipeline	fillSprite → draw into sprite → pushSprite. All drawing in RAM, invisible. Single atomic push to display. No flicker.
Sprite RAM	width × height × 2 bytes. Create once in setup, reuse across zones. Never delete and recreate in loop.
Zone layout	Define zone Y/H constants. Draw static chrome once with tft. Update dynamic zones with sprite. Same discipline as T05 OLED, larger canvas.

What's next

- T09 — Touch input: the XPT2046 touch controller on the TFT module adds a completely new input method. Calibration, touch zones, and integrating touch events with the dashboard layout are the natural next step.
- T10 — SD card and file I/O: with the TFT display working, loading a BMP splash screen from the SD card on startup is a satisfying next milestone — and introduces file I/O for data logging.

E4 Educator v2.0 · T08: TFT display and sprite pipeline · Walark Electronics · Fundrum Engineering · May 2026